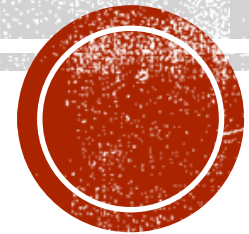
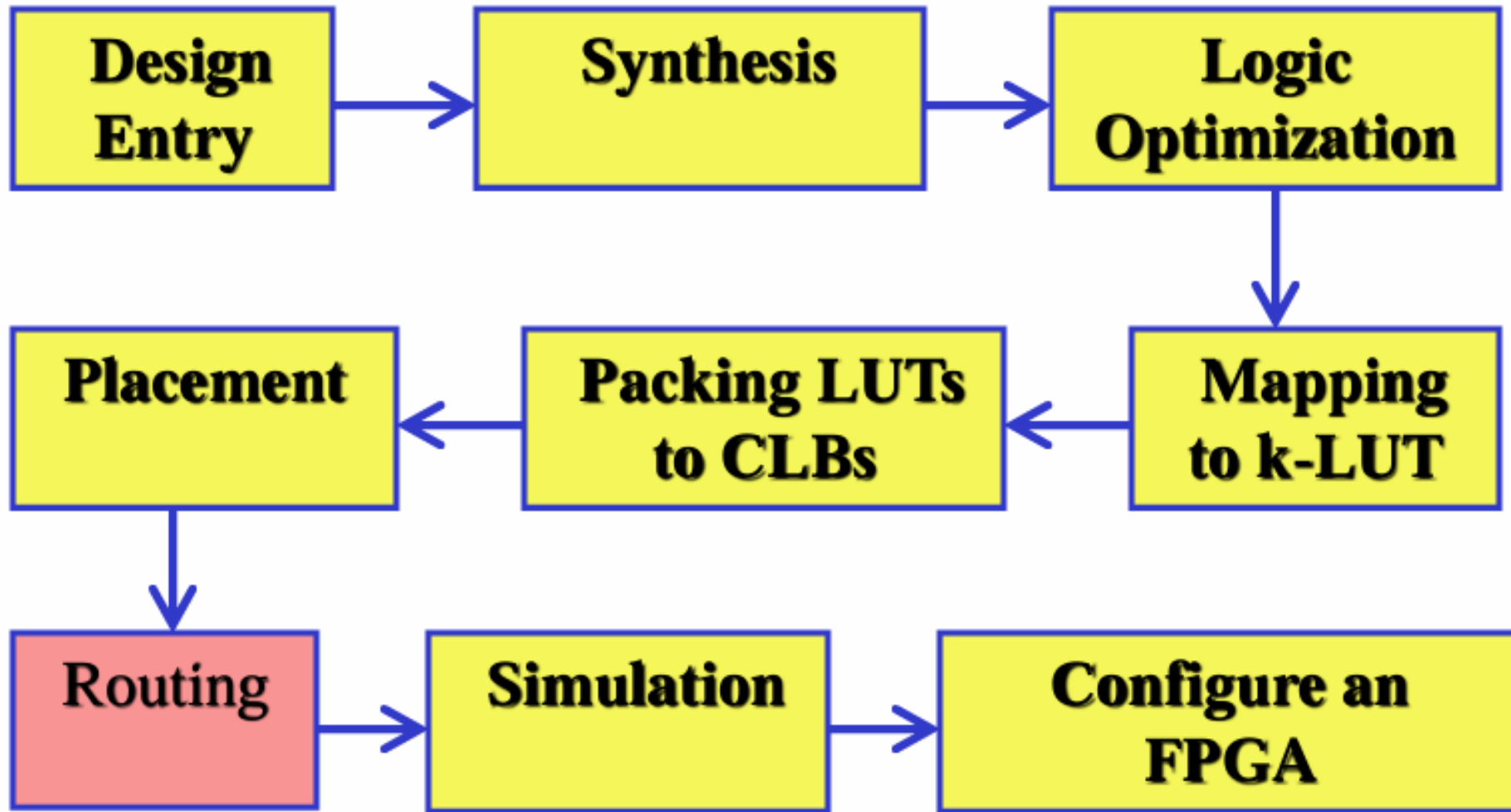


RECONFIGURABLE COMPUTING

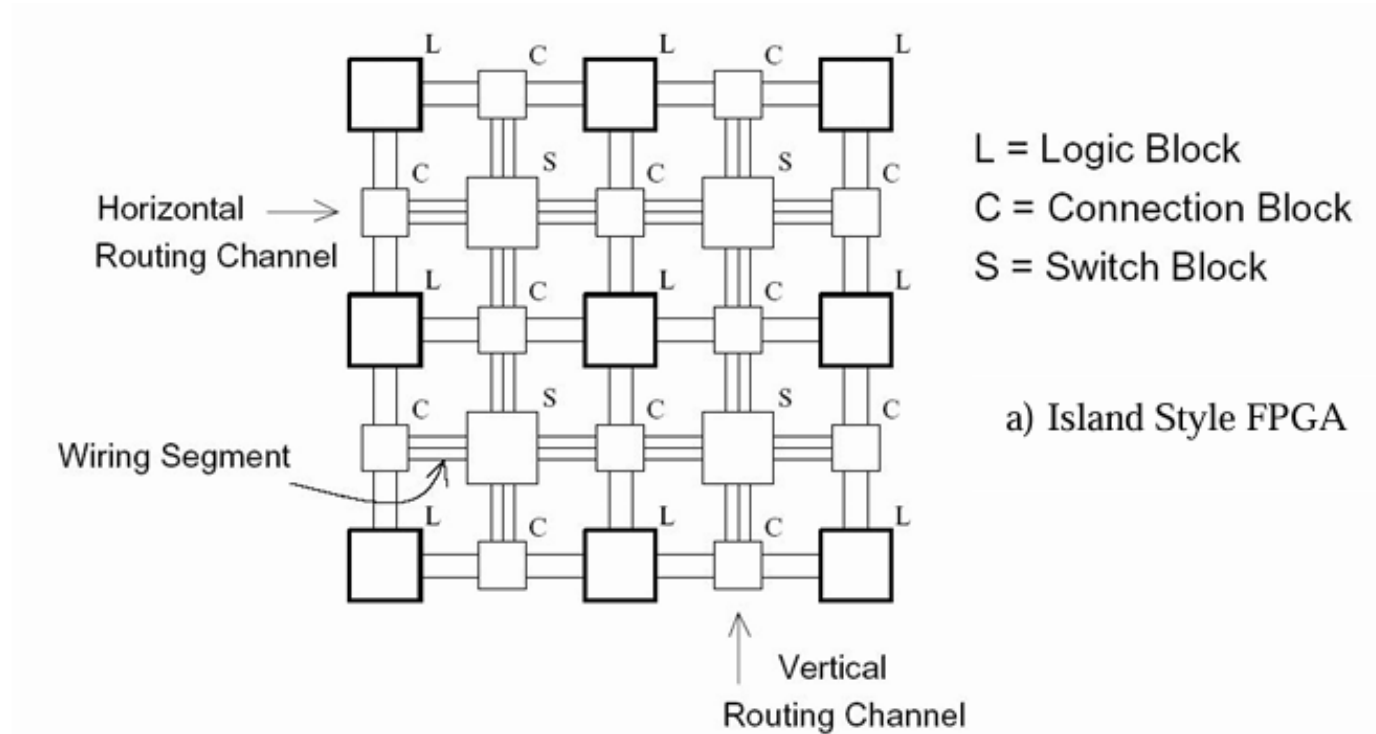
CS G553: Reconfigurable computing

Dr. Meetha V Shenoy



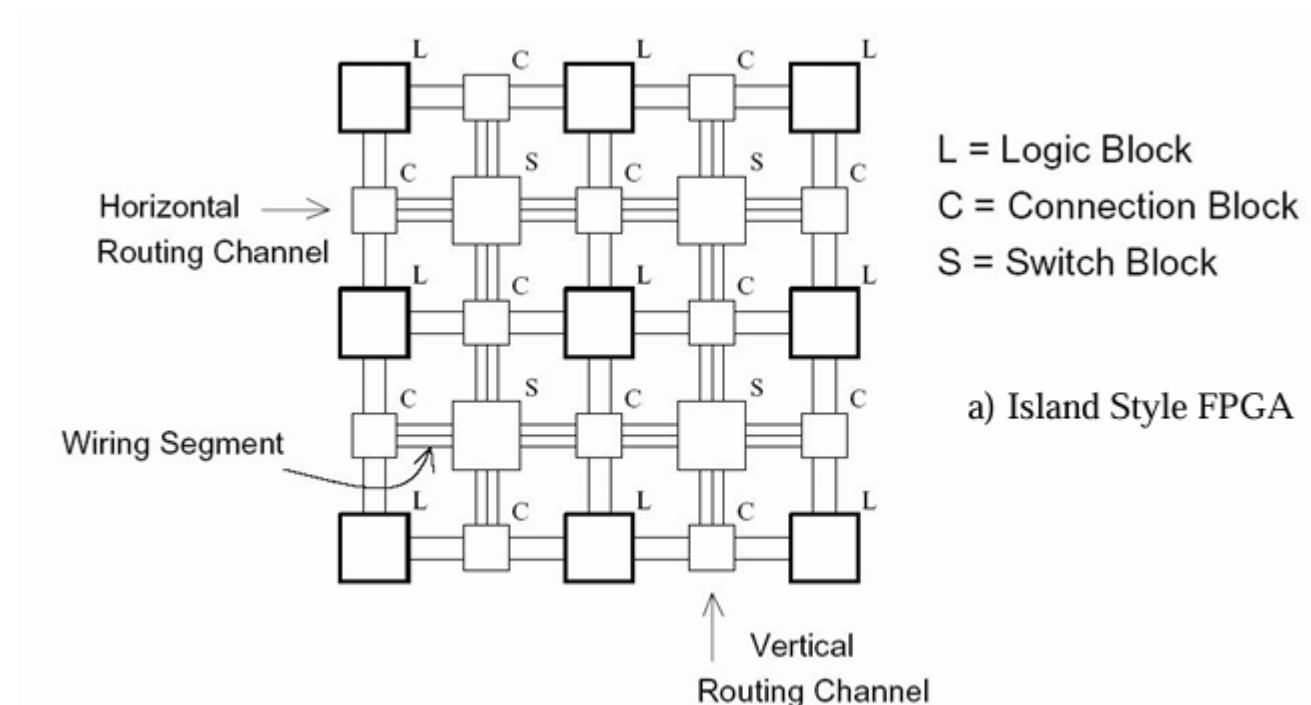


ISLAND STYLE FPFA



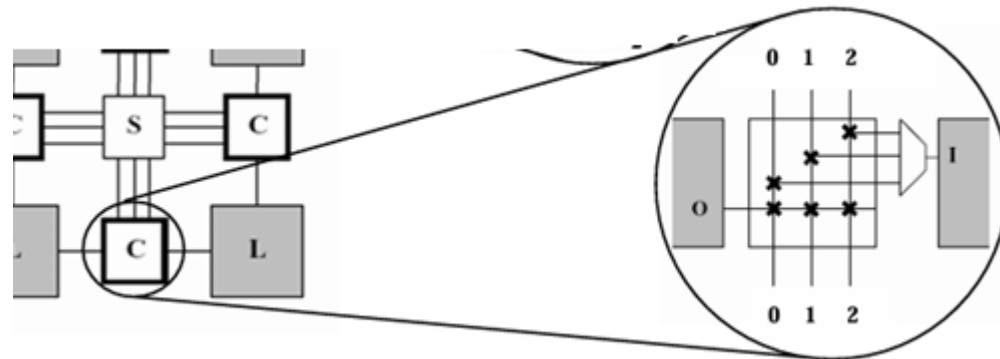
XILINX FPGA

- Xilinx FPGAs consist of a two-dimensional array of programmable blocks, called Configurable Logic Blocks – CLB's , with horizontal and vertical routing channels between CLB's rows and columns.



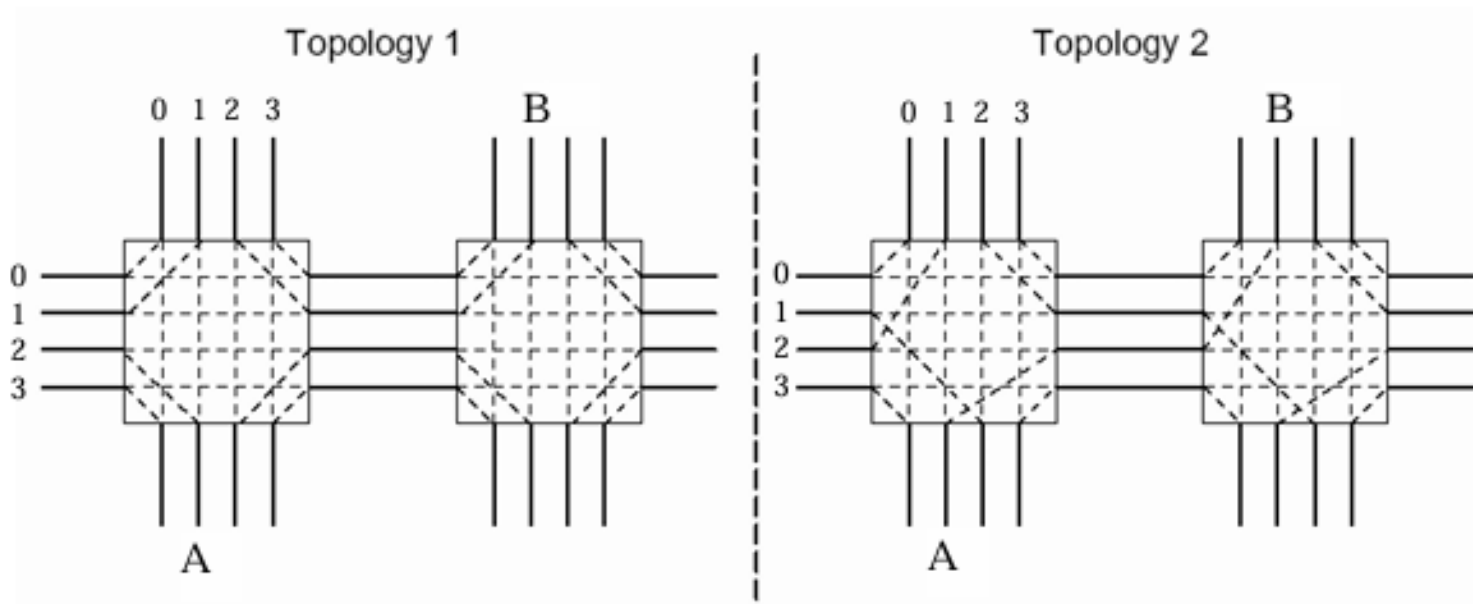
RESOURCES

- The C boxes connect the channel wires with the input and output pins of the CLBs. **Property:** flexibility, F_c , which is the number of wires that each logic block pin can connect to.



RESOURCES

- Switch boxes: The S boxes allow wires to switch between vertical and horizontal wires.



Switch boxes that connect only tracks in the same domain, i.e. 0-0, 1-1, are called **planar or subset switch boxes**.

Switch boxes that allow connection to any other domains, i.e. 0-3, 1-2, are called **Wilton switch boxes**



MAZE ROUTING- KEY COMPONENTS

- **Grid Representation:**

- The FPGA routing architecture is represented as a grid of routing resources.
- Each node in the grid corresponds to a switch, wire segment, or pin.

- **Source and Destination:**

- The **source** is the starting node (e.g., an output pin).
- The **destination** is the target node (e.g., an input pin).



MAZE ROUTING- KEY COMPONENTS

- Grid Representation:**

- The FPGA routing architecture is represented as a grid of routing resources.
- Each node in the grid corresponds to a switch, wire segment, or pin.

- Source and Destination:**

- The **source** is the starting node (e.g., an output pin).
- The **destination** is the target node (e.g., an input pin).

- Obstacles:**

- Used or unavailable routing resources act as obstacles.
- These could be due to other nets already routed or constraints in the design.



		s					
					d		

(a) Basic Grid

5	4	3	4	5	6	7	
4	3	2	3	4	5	6	7
3	2	1 _s	2	3	4	5	6
4	3	2	3	4	5	6	7
5	4	3	4	5	6	7	
6	5	4	5	6	7 _d		
7	6	5	6	7			
	7	6	7				

(d) Expansion – Destination reached

		2					
	2	1 _s	2				
		2					
					d		

(b) Expansion

5	4	3	4	5	6	7	
4	3	2	3	4	5	6	7
3	2	1 _s	←	←	←	5	6
4	3	2	3	4	↑	6	7
5	4	3	4	5	↑	7	
6	5	4	5	6	7 _d		
7	6	5	6	7			
	7	6	7				

(e) Backtracking

		3					
	3	2	3				
3	2	1 _s	2	3			
	3	2	3				
		3					
					d		

(c) Expansion cont.

		s					
						d	

(f) A minimum length route



MAZE ROUTING- KEY COMPONENTS

- Grid Representation:**

- The FPGA routing architecture is represented as a grid of routing resources.
- Each node in the grid corresponds to a switch, wire segment, or pin.

- Source and Destination:**

- The **source** is the starting node (e.g., an output pin).
- The **destination** is the target node (e.g., an input pin).

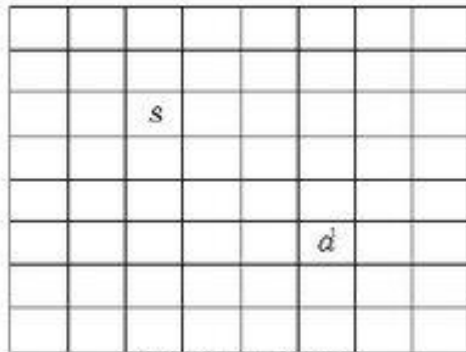
- Obstacles:**

- Used or unavailable routing resources act as obstacles.
- These could be due to other nets already routed or constraints in the design.

- Cost Function:**

- A cost function determines the quality of the path:
 - Wire Length:** Minimize the number of grid cells in the path.
 - Congestion:** Avoid congested areas to minimize delays.
 - Timing:** Ensure the path meets timing constraints.

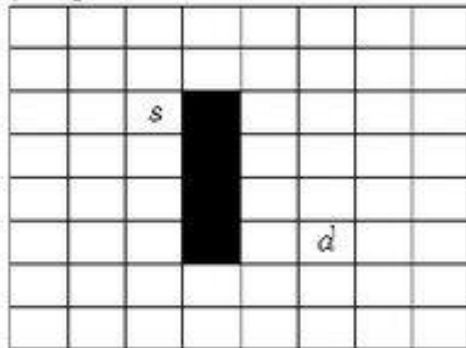




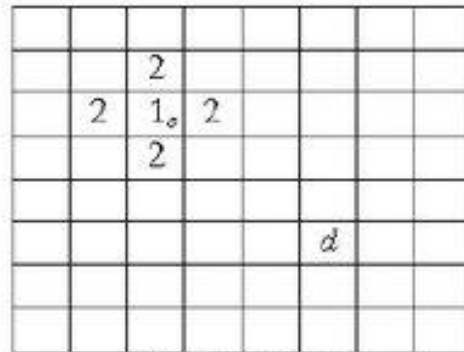
(a) Basic Grid



(d) Expansion - Destination reached



(g) Obstructions present ■



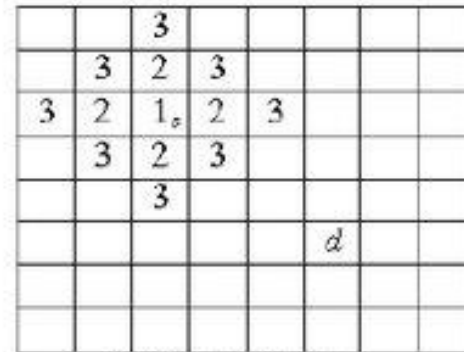
(b) Expansion



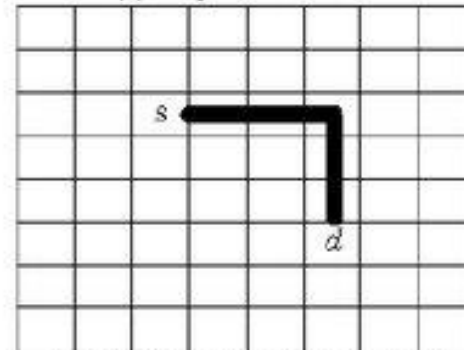
(e) Backtracking



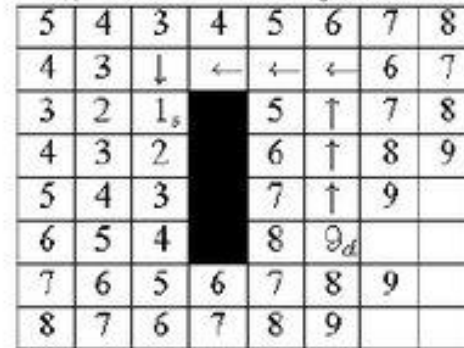
(h) Expansion with obstructions



(c) Expansion cont.



(f) A minimum length route



(i) Backtracking with obstructions



MAZE ROUTING IN FPGA

- **Maze Routing Algorithm :** The algorithm is a variant of the breadth-first search (BFS) or Dijkstra's shortest path algorithm.
- **1. Initialization:**
 - Define the grid with all routing resources.
 - Mark used resources as blocked.
- **2. Wave Expansion:**
 - Start from the source node and propagate outward (like a "wave").
 - Mark visited nodes to avoid revisiting.
 - Assign a cost to each node based on:
 - Distance from the source.
 - Congestion levels (penalize crowded areas).
- **3. Path Backtracking:**
 - Once the destination is reached, backtrack to the source to determine the optimal path.
 - Store this path as the final route.
- **4. Optimization:**
 - Adjust the path to reduce congestion or improve timing if needed.
 - Consider re-routing other nets if conflicts arise.

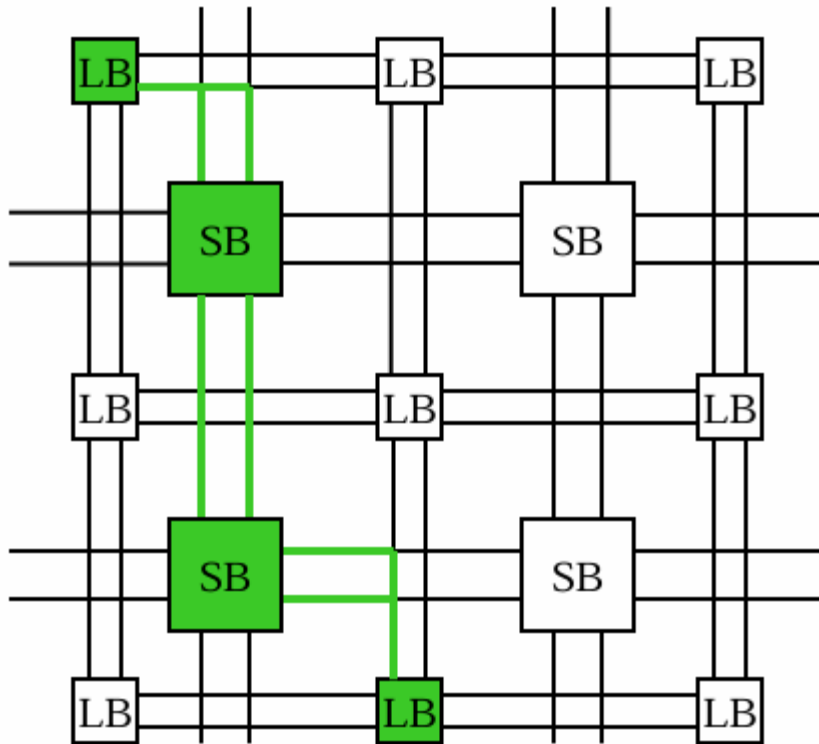


FPGA ROUTING

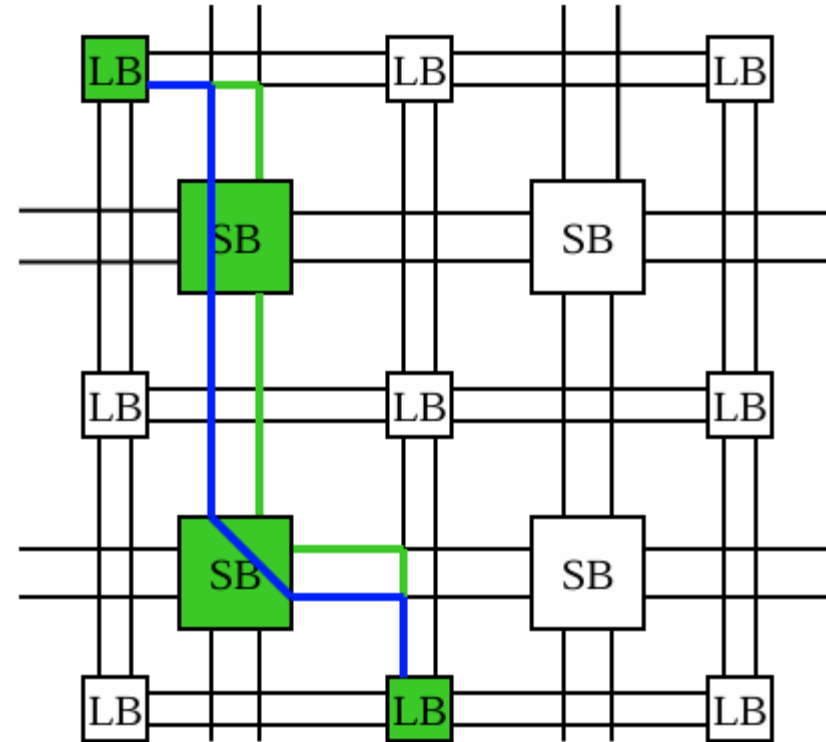
- More challenging than ASIC due to the constrained routing fabric.
- Routing is generally divided into two phases:
- **Global routing:** Selects the general path through the chip but does not determine the exact wire segments to be used.
- **Detailed routing:** selects the exact set of wires to be used for each connection



□ Global routing



□ Detailed routing



Steps in the **Pathfinder Algorithm**

▪ **1. Initialization**

- The FPGA is modeled as a routing graph, where:
 - Nodes represent routing resources (e.g., wires, switches, or pins).
- Each net has a source and one or more sinks to connect.

▪ **2. Initial Routing**

- Each net is routed independently, typically using a shortest-path algorithm like Dijkstra's or A*.
- During this step, congestion (resource sharing) is ignored.

▪ **3. Congestion Analysis**

- After the initial routing, resources used by multiple nets are identified.
- Congestion is calculated as the number of nets using a resource beyond its capacity.



- Steps in the **Pathfinder Algorithm**.
- **4. Cost Function Adjustment**
- The cost of each resource is updated to reflect congestion:

$$C_{\text{total}} = C_{\text{base}} + C_{\text{history}} + C_{\text{present}}$$

C_{base} : Base cost of the resource.

C_{history} : A penalty for resources that have been congested in previous iterations.

C_{present} : A penalty for current congestion.

$$\text{congestion} = \max(0, \text{usage} - \text{capacity})$$



- **5. Rerouting**

- Nets sharing congested resources are rerouted using the updated cost function.
- Nets are routed one at a time to ensure progress.

- **6. Iteration**

- Steps 3–5 are repeated for several iterations.
- Over time, congestion penalties (history costs) accumulate, discouraging overused resources.

- **7. Termination**

- The algorithm terminates when:
 - All resources are within capacity (no congestion).
 - Or, a fixed number of iterations are completed, yielding a near-optimal solution.

